

FRED TALKS

29th April 2026

CONTENTS

How big are our embeddings now and why?

★♥☆ VICKI BOYKIS ★♥☆ · 1697 WORDS

Zen of AI Coding

YOAV AVIRAM · 1912 WORDS

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 1975 WORDS

New work with @AlecRad and @DavidDuvenaud:

NICK LEVINE · 54 WORDS

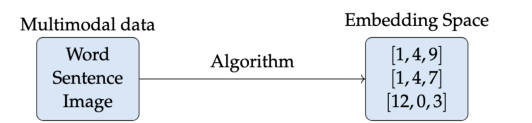
How big are our embeddings now and why?

★♥☆ VICKI BOYKIS ★♥☆ · 10 SEP 2025 · [SOURCE](#)

A few years ago, I wrote [a paper on embeddings](#). At the time, I wrote that 200-300 dimension embeddings were fairly common in industry, and that adding more dimensions during training would create diminishing returns for the effectiveness of your downstream tasks (classification, recommendation, semantic search, topic modeling, etc.)

I wrote the paper to be resilient to changes in the industry since it focuses on fundamentals and historical context rather than libraries or bleeding edge architectures, but this assumption about embedding size is now out of date and worth revisiting in the context of growing embedding dimensionality and embedding access patterns.

As a quick review, embeddings are compressed numerical representations of a variety of features (text, images, audio) that we can use for machine learning tasks like search, recommendations, RAG, and classification. The size of the embedding is how many features our item has.



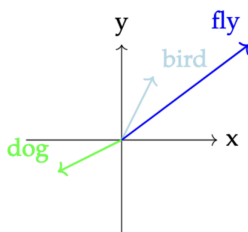
For example, let's say we have two butterflies. We can compare them among many dimensions, including wingspan, wing color, number of antennae. Let's say we have a 3-dimensional feature for a butterfly, it might look like this.

	wing_color	wing_span	antennae
butterfly_1	blue	10 cm	1
butterfly_2	red	20cm	2
butterfly_3	blue	15cm	1

Doing a visual vibe scan, we can eyeball the data and see that `butterfly_1` and `butterfly_3` are more similar to each other than to `butterfly_2` because their features are closer together.

But butterflies are 3-dimensional animals and some of these features are numerical. When we talk about embeddings in industry these days, we generally mean trying to understand the properties of text, so how does this concept work with words? We can't directly compare words like "bird" and "butterfly" or "fly" in a given text but we can compare their numerical representations if we map them into the same shared space. We can see that "bird" and "flying" are more similar to each other than to "dog" through their numerical representations.

Intuitively, we know this is true, but how do we artificially create these relationships?

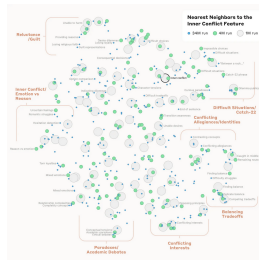


There are different ways of creating text embeddings from a given word, all of which rely on analyzing that word in relation to the words around it in a given corpus of data. We can use traditional count-based approaches like TF-IDF based on term frequency in documents, or statistical approaches like PCA or LSA.

With the advent of deep learning models, we started learning representations generated from models like Word2Vec that maximized the probability that a left-out word would be next to other given words in the training dataset.

When we learn the embedding representation of a given word using probabilistic models, we are comparing how similar these words are to other words. Each feature is not an explicit feature like "wing color", but rather a vibe-based latent representation in the latent space that doesn't have a clear explanation.

For example, one dimension might be "this word is an action word" or maybe "this word is related to other words about food", but we generally don't know exactly what the model thinks each feature represents. In fact, this is a fascinating area of study we are just starting to understand how these latent representations work through ideas like control vectors, a concept that Anthropic explored in the famous Golden Gate Claude paper.



When we train a model, embedding size is initialized as a hyperparameter before model training, and we iterate on the size depending on our downstream evaluations after training. Picking the right hyperparameter is (alchemy) a combination of art and science and depends on optimizing training throughput, final embedding storage size, and the performance of the embedding on your downstream task both qualitatively and wrt to latency of performing search on embeddings of different sizes.

Since previous generations of models were smaller and trained in-house, hyperparameters were usually not published by companies, and as such, there was no standard agreement on embedding size. We generally, as an industry, understood that somewhere around 300 dimensions for a given embedding model might be enough to compress all the nuance of a given textual dataset. 300 was the number of dimensions typically used by earlier models like Word2Vec and GloVE.

After the publication of the attention paper BERT was released in 2018. This model's architecture introduced embeddings of 768 dimensions. Although previous RNN and LSTM models had been trained on GPUs, BERT was one of the first larger embedding models to be trained on GPUs (and TPUs), which meant that GPU optimization now became increasingly important.

The key behind training Transformer models efficiently is the ability to efficiently move data onto GPU and parallelize their matrix multiplication operations between several pipelines, aka attention heads, where each attention head can focus on understanding and defining a different part of the embedding feature space. As such, the embedding needs to be able to be partitioned evenly between the number of attention heads.

BERT has 12 attention heads, so 768 dimensions was selected from a combination of trial and error and efficiently parallelizing computation to “attend” to different parts of the feature space, meaning that, in model training, each head operates on a 64-dimensional subspace of the original 768-dimensional input embedding. This itself comes from the Transformer paper, where each sub-embedding size per head is commonly chosen as 64.

As a result, many BERT-style models, and related model families, used 768 as a baseline for embedding dimensions. Although it had a much larger training dataset, GPT-2 also implemented 768. And, although CLIP uses an embedding size of **768** as derived from the Vision Transformer architecture that CLIP uses for its image encoder, and for consistency, the text encoder also uses this dimension size^[1].

Even though training cycles for BERT were fairly small (4 days for the original BERT model) compared to the months-long pretraining processes that LLMs require these days, it was still hard computationally to infer these embedding sizes, even with GPU optimizations. For BERT, for example,

Finding the most similar pair in a collection of 10,000 sentences req

So in 2019, UKP created and optimized a model, SBERT, focusing on sentence-level representations, which unlocked faster inference, and Minilm became the standard baseline model for document-level chunk embeddings at 384 dimensions and still performs extremely well for a number of tasks.

What else changed was the rise of the open-source HuggingFace platform where model artifacts could be shared. Previously, they were either hosted in undiscoverable repos in GitHub or locked away in scientific repositories missing metadata. The rise of HuggingFace as a platform and the `transformers` library as a centralized API into training and inference for PyTorch-based models unlocked a level of standardization: now, many more people could simply download and replicate the existing models instead of rebuilding arcane research code from scratch. This led to much more standard embedding and architecture sizes used both in academia and industry.

Although 768 held for a fairly long time, with upwards pressure from market competition on model sizes as the result of the release of GPT-2 (the backbone of ChatGPT), we started seeing standard embedding sizes increase.

Part of this was the fact that companies no longer had to infer their own embeddings. Previously, embeddings had been custom-learned in labs or in companies whose core competency was processing large amounts of information that could be culled in retrieval through search or recommendation.

But now, with the advent of ChatGPT and API-based model availability, embeddings became a commodity available with a GET request, and the most popular embeddings became OpenAI's, which used 1536 dimensions, in line with GPT-3, which used much more training data than any previous model. (570GB versus GPT-2 which was 40GB, and 96 attention heads.)

It used to be the case that people only learned or fine-tune their own embeddings, but now all of the major AI providers have their own hosted sets of embeddings. Particularly common ones are OpenAI, with Cohere and Nomic close behind. Google also recently released embeddings for Gemini, with both API and local versions being available.

In addition to standardization via HuggingFace and APIs, MTEB, which benchmarks embeddings publicly, has grown as a resource where you can compare embedding models.

If we look at MTEB, the embedding benchmark today, we can see embedding sizes anywhere from our classic 768 to 4096 and beyond (note all of them are neatly divisible by 2, in line with architectural constraints)

Rank (Des.)	Model	Zero-shot	Embedding S.
1	gemini-embedding-001	99%	3072
2	Qwen2-Embedding-8B	99%	4096
3	Qwen2-Embedding-4B	99%	2560
4	Qwen2-Embedding-1.5B	99%	1024
5	Llama-Embed-Mistral	99%	4096
6	gte-Qwen2-7B-Instruct	▲ NA	3072
7	multilingual-e5-large-instruct	99%	1024
8	SFR-Embedding-Mistral	96%	4096
9	text-multilingual-embedding-002	99%	768
10	GritLM-7B	99%	4096
11	GritLM-Bv7B	99%	4096

Finally, another consideration has been the change in the landscape with vector databases, which used to be huge, are now becoming more commoditized features in software and platforms like Postgres, S3, and Elasticsearch, leading to less out of the box necessity in vector storage and performance tuning.

With the constant upward pressure on embedding sizes not limited by having to train models in-house, it's not clear where we'll slow down: Qwen-3, along with many others is already at 4096. It does appear that we are beginning to trim down on growth. OpenAI implemented a concept called matryoshka representation learning that trains embeddings with the “most important” concepts first, and additional embeddings adding incremental gains, meaning that an embedding learned in 1024 dimensions might be just as useful in 64, as long as the first 64 dimensions compress most of the information efficiently, and also making sure we re-normalize them. There are also research that indicates that not all embeddings are necessary in retrieval + search tasks, and that we can in fact, in some cases, truncate 50% of them anyway.

It's been fascinating to watch the rise of dimensionality and the constant struggle in tradeoffs between creating ever-larger models and then the need for those embedding sizes to perform at inference and storage time, aka continuously coming up against the age-old machine learning tradeoff of recall versus precision, and the engineering tradeoff of hardware limitations versus software versus business considerations.

As these architectures have matured and internal models have become inference points of public-facing paid APIs, embeddings have gone from a mysterious byproduct of internal machine learning systems in companies with lots of data to a commodity used across many AI-powered applications across numerous application stacks.

All in all, it's been so much fun watching this space evolve, even if it means it looks like I'll be having to update my paper over and over again.

Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · [SOURCE](#)

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to [Zen of Python](#) by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead
2. Code is cheap
3. Refactoring easy
4. So is repaying technical debt
5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack
8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world

11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents
16. Anticipate modes of failure

Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

All bugs are shallow

Linus's law states that “given enough eyeballs, all bugs are shallow.” Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How “dense” are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We've built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is trickier then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

Agent-first software engineering

We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.

Agent-first product strategy

I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of [WhiteBoar](https://whiteboar.it), where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboar.it.

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 14 MAR 2026 · [SOURCE](#)

It's a [common position](#) among software engineers that big egos have no place in tech¹. This is understandable - we've all worked with some insufferably overconfident engineers who needed their egos checked - but I don't think it's correct. In fact, I don't know if it's possible to survive as a software engineer in a large tech company without some kind of big ego.

However, it's more complicated than "big egos make good engineers". The most effective engineers I've worked with are simultaneously high-ego in some situations and surprisingly low-ego in others. What's going on there?

Engineers need ego to work in large codebases

Software engineering is shockingly humbling, even for experienced engineers. There's a reason this joke is so popular:



The minute-to-minute experience of working as a software engineer is dominated by *not knowing things* and *getting things wrong*. Every time you sit down and write a piece of code, it will have several things wrong with it: some silly things, like missing semicolons, and often some major things, like bugs in the core logic. We spend most of our time fixing our own stupid mistakes.

On top of that, even when we've been working on a system for years, we still don't know that much about it. I wrote about this at length in *Nobody knows how large software products work*, but the reason is that big codebases are just that complicated. You simply can't confidently answer questions about them without going and doing some research, even if you're the one who wrote the code.

When you have to build something new or fix a tricky problem, it can often feel straight-up impossible to begin, because good software engineers know just how ignorant they are and just how complex the system is. You just have to throw yourself into the blank sea of millions of lines of code and start wildly casting around to try and get your bearings.

Software engineers need the kind of ego that can stand up to this environment. In particular, they need to have a firm belief that they *can* figure it out, no matter how opaque the problem seems; that if they just keep trying, they can break through to the pleasant (though always temporary) state of affairs where they understand the system and can see at a glance how bugs can be fixed and new features added².

Engineers need ego to work in big tech companies

What about the non-technical aspects of the job? Nobody likes working with a big ego, right? Wrong. Every great software engineer I've worked with in big tech companies has had a big ego - though as I'll say below, in some ways these engineers were surprisingly low-ego.

You need a big ego to take positions. Engineers love being non-committal about technical questions, because they're so hard to answer and there's often a plausible case for either side. However, as I keep saying, engineers have a duty to take clear positions on unclear technical topics, because the alternative is a non-technical decision maker (who knows even less) just taking their best guess. It's scary to make an educated guess! You know exactly all the reasons you might be wrong. But you have to do it anyway, and ego helps *a lot* with that.

You need a big ego to be willing to make enemies. Getting things done in a large organization means making some people angry. Of course, if you're making lots of people angry, you're probably screwing up: being too confrontational or making obviously bad decisions. But if you're making a large change and one or two people are angry, that's just life. In big tech companies, any big technical decision will affect a few hundred engineers, and one of them is bound to be unhappy about it. You can't be so conflict-averse that you let that stop you from doing it, if you believe it's the right decision. In other words, you have to have the confidence to believe that you're right and they're wrong, even though technical decisions always involve unclear tradeoffs and it's impossible to get absolute certainty.

You need a big ego to correct incorrect or unclear claims. When I was still in the philosophy world, the Australian logician Graham Priest had a reputation for putting his hand up and stopping presentations when he didn't understand something that was said, and only allowing the seminar to continue when he felt like he understood. From his perspective, this wasn't rude: after all, if *he* couldn't understand it, the rest of the audience probably couldn't either, and so he was doing them a favor by forcing a more clear explanation from the speaker.

This is obviously a sign of a big ego. It's also a trait that you need in a large tech company. People often nod and smile their way past incorrect technical claims, even when they suspect they might be wrong - assuming that they've just misunderstood and that somebody else will correct it, if it's truly wrong. **If you are the most senior engineer in the room, correcting these claims is your job.**

If everyone in the room is so pro-social and low-ego that they go along to get along, decisions will get made based on flatly incorrect technical assumptions, projects will get funded that are impossible to complete, and engineers will burn weeks or months of their careers vainly trying to make these projects work. You have to have a big enough ego to think "actually, I think I'm right and everyone in this room is confused", even when the room is full of directors and VPs.

Sometimes you need to put your ego aside

All of this selects for some pretty high-ego engineers. But in order to actually *succeed* in these roles in large tech companies, you need to have a surprisingly low ego at times. **I think this is why really effective big tech engineers are so rare: because it requires such a delicate balance between confidence and diffidence.**

To be an effective engineer, you need to have a towering confidence in your own ability to solve problems and make decisions, even when people disagree. But you also need to be willing to instantly subordinate your ego to the organization, when it asks you to. At the end of the day, your job - the reason the company pays you - is to execute on your boss's and your boss's boss's plans, whether you agree with them or not.

Competent software engineers are allowed quite a lot of leeway about *how* to implement those plans. However, they're allowed almost no leeway at all about the plans themselves. In my experience, being confused about this is a common cause of burnout³. Many software engineers are used to making bold decisions on technical topics and being rewarded for it. Those software engineers then make a bold decision that disagrees with the VP of their organization, get immediately and brutally punished for it, and are confused and hurt.

In fact, **sometimes you just get punished and there's nothing you can do.** This is an unfortunate fact of how large organizations function: even if you do great technical work and build something really useful, you can fall afoul of a political battle fought three levels above your head, and come away with a *worse* reputation for it. Nothing to be done! This can be a hard pill to swallow for the high-ego engineers that tend to lead really useful technical projects.

You also have to be okay with having your projects cancelled at the last minute. It's a very common experience in large tech companies that you're asked to deliver something quickly, you buckle down and get it done, and then right before shipping you're told "actually, let's cancel that, we decided not to do it". This is partly because the decision-making process can be pretty fluid, and partly because many of these asks originate from off-hand comments: the CTO implies that something might be nice in a meeting, the VPs and directors hustle to get it done quickly, and then in the next meeting it becomes clear that the CTO doesn't actually care, so the project is unceremoniously cancelled⁴.

Final thoughts

Nobody likes to work with a bully, or with someone who refuses to admit when they're wrong, or with somebody incapable of empathy. But you really do need a strong ego to be an effective software engineer, because software engineering requires you to spend most of your day in a position of uncertainty or confusion. If your ego isn't strong enough to stand up to that - if you don't believe you're good enough to power through - you simply can't do the job.

This is particularly true when it comes to working in a large software company. Many of the tasks you're required to do (particularly if you're a senior or staff engineer) require a healthy ego. However, there's a kind of catch-22 here. If it insults your pride to work on silly projects, or to occasionally "catch a stray bullet" in the organization's political fights, or to have to shelve a project that you worked hard on and is ready to ship, you're too high-ego to be an effective software engineer. But if you can't take firm positions, or if you're too afraid to make enemies, or you're unwilling to speak up and correct people, you're too low-ego.

Engineers who are low-ego in general can't get stuff done, while engineers who are high-ego in general get slapped down by the executives who wield real organizational power. The most successful kind of software engineer is therefore a chameleon: low-ego when dealing with executives, but high-ego when dealing with the rest of the organization⁵.

1. What do I mean by "ego", in this context? More or less the colloquial sense of the term: a somewhat irrational self-confidence, a tendency to believe that you're very important, the sense that you're the "main character", that sort of thing

↩

2. Why is this "ego", and not just normal confidence? Well, because of just how murky and baffling software problems feel when you start working on them. You really do need a degree of confidence in yourself that feels unreasonable from the inside. It should be obvious, but I want to explicitly note that you don't *just* need ego: you also have to be technically strong enough to actually succeed when your ego powers you through the initial period of self-doubt.

↩

3. I share the increasingly-common view that burnout is not caused by working too hard, but by hard work unrewarded. That explains why nothing burns you out as hard as being punished for hard work that you expected a reward for.

↩

4. It's more or less exactly this scene from Silicon Valley.

↩

5. This description sounds a bit sociopathic to me. But, on reflection, it's fairly unsurprising that competent sociopaths do well in large organizations. Whether that kind of behavior is worth emulating or worth avoiding is up to you, I suppose.

↩

New work with @AlecRad and @DavidDuvenaud:

NICK LEVINE · 28 APR 2026 · [SOURCE](#)

New work with [@AlecRad](#) and [@DavidDuvenaud](#):

Have you ever dreamed of talking to someone from the past? Introducing talkie, a 13B model trained only on pre-1931 text.

Vintage models should help us to understand how LMs generalize (e.g., can we teach talkie to code?). Thread:

[VIDEO OMITTED]